Citizen Voice: Bilingual Review Intelligence for UAE Government Mobile Apps

Youssef Hajri
Student ID: 1425158
MAAI902 Natural Language Processing & LLMs
University of Wollongong in Dubai

October 2026

Abstract

People in the UAE review government apps such as UAE PASS, DubaiNow, RTA Dubai and DEWA every day, in English and in Arabic, and there are far too many reviews for anyone to read. For this project I built Citizen Voice, an NLP and LLM application that collects these reviews automatically and tells an app team what is broken, how upset people are, which release caused it and how to reply. The pipeline covers preprocessing for both languages (NLTK, spaCy and CAMeL Tools), dependency-based aspect and relation extraction, four text representations (BoW, TF-IDF, Word2Vec and the multilingual BGE-M3), five sentiment approaches including a LoRA fine-tune of XLM-RoBERTa, emotion detection, three NER systems, issue discovery by clustering, a statistical release-regression detector, and a retrieval-augmented assistant whose answers are checked by a second LLM. I evaluated the methods on 5,271 real reviews and made everything available in an interactive Streamlit app. The project is linked to the Smart Government priority of the UAE National AI Strategy 2031.

Contents

1 Introduction **4**

2 Related Work **4**

3 Data **4**

4 Methodology **5**
4.1 Bilingual preprocessing 6
4.2 Linguistic analysis 6
4.3 Text representations 6
4.4 Sentiment, fine-tuning and emotion 7
4.5 Named entities and relations 7
4.6 Issue discovery and release regressions 7
4.7 Prompt engineering, RAG and faithfulness 8

5 Results **8**

6 The Citizen Voice Application **10**
6.1 The application in use 11

7 Discussion **16**

8 Limitations and Ethics **17**

9 Conclusion **17**

List of Figures

1 Star ratings by app, language mix and review length. 5
2 Left: POS distribution in complaints and praise. Right: most frequent bigrams per language and class. 7
3 Normalised confusion matrices: TF-IDF with LR, routed pre-trained models, LoRA fine-tuned XLM-R. 9
4 Emotions in complaints by app, and average mood on the valence-arousal plane. 9
5 Complaint rate per version for the most reviewed app (red: flagged). 10
6 Top of the results dashboard exported by the pipeline. 11
7 Overview tab: key numbers, predicted sentiment per app and the emotions in complaints. 12
8 Issues tab: complaints clustered with BGE-M3, with the share of Arabic reviews and the class-based TF-IDF keywords of each issue. 12
9 Releases tab: versions flagged by the two-proportion z-test after the Benjamini-Hochberg correction, with the issue that grew the most. 13
10 Search tab: the English query “OTP code never arrives” returns English and Arabic reviews about the same problem. 13
11 Ask Citizen Voice tab: a question in English or Arabic is answered by the RAG assistant from the retrieved reviews, with citations and a faithfulness check by a second LLM call. 14

12 Analyze a review tab: any new review can be analysed step by step, from cleaning and tokens to entities, sentiment, emotion and urgency. 14
13 Triage and Reply tab: unanswered reviews ranked by the urgency score, ready for a bilingual reply draft. 15

List of Tables

1 Government apps targeted by the collector (Google Play, UAE store). 5
2 Vocabulary size (distinct word types) after each preprocessing step. 6
3 Text representations (same logistic-regression classifier, test set). 8
4 Cross-lingual retrieval with parallel English and Arabic queries (mean over 5 pairs). 8
5 Sentiment analysis on the test set (star labels as ground truth). 8
6 Prompt engineering on a balanced sample (same LLM, different prompts). . 9
7 Largest automatically discovered issues (13 clusters in total). 9
8 Flagged release regressions after the Benjamini-Hochberg correction (10 of 65 versions). 10
9 RAG assistant: faithfulness of the answers (share of supported claims) with and without retrieval. 10

1 Introduction

Most government services in the UAE are now used through mobile apps, so app store reviews have become one of the most direct ways residents give feedback. They are free and arrive every day, but they are also short, noisy and written in English, Modern Standard Arabic, Gulf dialect or a mix of them. I wanted to answer three questions that a digital-services team would actually ask: what is failing (OTP codes, face recognition, payments), how serious the frustration is, and when it started, for example after an update.

The UAE National AI Strategy 2031 names smart government and Arabic language resources as priorities [21], so I decided from the start to treat Arabic as a full language in the system and not as an afterthought. My main contributions are:

- an automatic and incremental collector for English and Arabic Play Store reviews that does not store reviewer names;
- a bilingual NLP pipeline with preprocessing, POS tagging, dependency parsing, n-grams, embeddings, sentiment, emotion, NER and relation extraction, and a comparison of the methods on the same data;
- three applied components: cross-lingual issue discovery, a release regression detector, and a RAG assistant with an automatic faithfulness check;
- a Streamlit application and a results dashboard so that non-technical staff can use the results.

2 Related Work

Mining app reviews is a known topic in software engineering, for example Guzman and Maalej [7] extract app features and the sentiment attached to them. Lexicon methods such as VADER [11] are fast and easy to explain but only work in English and ignore context. Transformers pre-trained on large corpora [4] and adapted to social media, such as Twitter-RoBERTa [16] and the multilingual XLM-T [1] built on XLM-R [3], are the usual choice today. For Arabic, CAMeL Tools [18] provides normalisation and tokenisation and CAMeLBERT [12] provides pre-trained models, including dialect sentiment and NER. Light stemming [13] and the ISRI root stemmer [20] are the classical Arabic stemmers. Multilingual embeddings such as BGE-M3 [2] place different languages in one vector space, and BERTopic [6] made it common to cluster embeddings and describe each cluster with class-based TF-IDF. LoRA [10] makes it possible to fine-tune large models by training small low-rank matrices. Retrieval-augmented generation [15] grounds LLM answers in documents, and LLM-as-a-judge [22] is a practical way to check them.

3 Data

Collection. The collector (`cvoice/collect.py`) uses `google-play-scraper` to download the newest reviews of UAE government apps from the UAE store, once with English and once with Arabic as the interface language. In total 6 apps were found. Each run is incremental, so reviews that were already collected are skipped, and if a package id is wrong the collector looks the app up with a store search. This means the collection can run on a schedule or be started from the app. For each review I keep the text, star rating, thumbs-up count, app version, date and the official reply if there is one.

How the scraping works. For every app in Table 1 the collector first checks that the package id exists on the UAE store. If it does not, it runs a store search with the app name and keeps the first result whose title matches, so a renamed or moved app is still

found. It then calls the `reviews` function of `google-play-scraper` with the country set to `ae`, the sort order set to newest first and the language set to English and then to Arabic, which returns the reviews written in that language. Reviews come in pages of 200 and the collector follows the continuation token until it has 600 new reviews per app and language or the store has no more. It waits half a second between pages and retries a failed request up to four times with a growing delay, so the free endpoint is not overloaded. Every review keeps its store id, and the ids already saved are skipped, which is what makes the collection incremental. The new rows are added to one Parquet file, duplicates and empty reviews are removed, and only then does the analysis pipeline run on the new reviews.

Table 1: Government apps targeted by the collector (Google Play, UAE store).

App	Package id	Service
UAE PASS	`ae.uaepass.mainapp`	National digital identity
DubaiNow	`com.deg.mdubai`	Dubai government services
RTA Dubai	`com.rta.rtadubai`	Roads, parking and transport
DEWA	`com.dewa.application`	Electricity and water
Dubai Police	`com.dubaipolice.app`	Police services and fines
UAEICP	`ae.gov.icp.smartservices`	Identity and residency
TAMM	`ae.abudhabi.tamm`	Abu Dhabi government services
MOHRE	`ae.gov.mohre`	Labour and work permits

Privacy. The reviews are public, but I never store reviewer names (only a short one-way hash) and the app only shows aggregated results. Generated replies are drafts that a person has to approve.

Labels. I used the star rating as a weak label: 1 or 2$\star$ is negative, 3$\star$ is neutral and 4 or 5$\star$ is positive. The final data set has 5,271 reviews (average rating 3.8, 28% complaints, 42% in Arabic, median length 3 words). Two things about this data affected every result later on. First, ratings are very polarised (mostly 1 or 5 stars), so neutral is rare and I report macro-F1 everywhere. Second, the reviews are very short: half of them have 3 words or fewer (for example mumtāz, “excellent”, or “not easy to login”), which gives any model very little context. I used one stratified split (by label and language) with 4216 training and 1055 test reviews for all experiments.

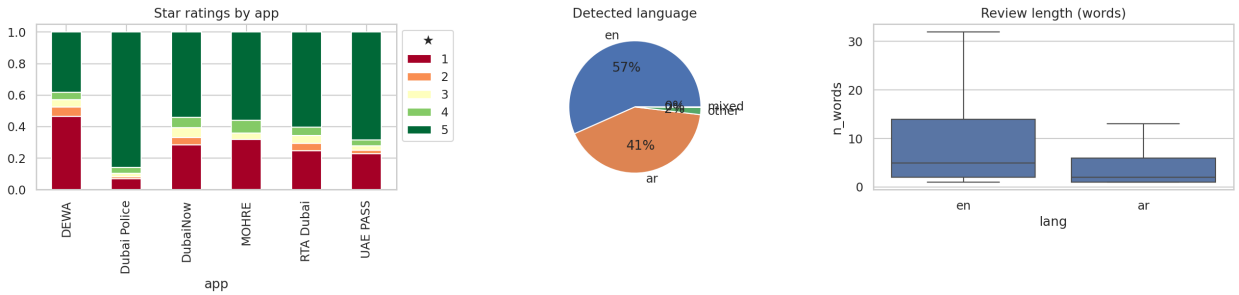


Figure 1: Star ratings by app, language mix and review length.

4 Methodology

The pipeline runs in this order: collection → language detection and preprocessing → linguistic analysis → sentiment, emotion and NER → BGE-M3 embeddings → issue clustering and release tests → LLM features → app and dashboard.

4.1 Bilingual preprocessing

I detect the language of each review from the share of Arabic and Latin letters (en, ar or mixed). English: I remove URLs, turn emoji into tokens (e.g. `emoji_enraged_face`), lower-case the text, shorten repeated letters, tokenise with NLTK, remove stop words and lemmatise with WordNet and spaCy. I also compare the Porter, Snowball and Lancaster stemmers. Arabic: I use CAMeL Tools to normalise the alef variants, alef maksura and teh marbuta, remove diacritics and tatweel, map Arabic-Indic digits to 0 to 9 and tokenise, then remove NLTK Arabic stop words and apply either the ISRI root stemmer or my own light stemmer, which removes one article or conjunction prefix and one suffix. In both languages I kept the negations out of the stop-word lists (not, never, lā, lam, lan, laysa...), because deleting them flips the sentiment. The transformer models get the original text, since their sub-word tokenisers were trained on raw text.

Table 2 shows what each step does to the vocabulary. Removing stop words barely changes the number of distinct words, because stop words are frequent but few. Stemming has the biggest effect, and much more in Arabic: the ISRI stemmer reduces 3,106 Arabic word types to 1,456, since many Arabic words are built on the same root with prefixes and suffixes. This also merges words with different meanings, so I used the lemmas, which keep more of the meaning, as the features for the classical models.

Table 2: Vocabulary size (distinct word types) after each preprocessing step.

Step	English	Arabic
Raw tokens	3,436	3,271
Without stop words	3,294	3,106
Stemmed (Porter / ISRI)	2,529	1,456
Lemmatised	2,688	2,644

4.2 Linguistic analysis

For English reviews I use spaCy [9] for POS tags and dependency trees. I wrote three dependency rules that extract aspect → opinion pairs: an adjectival modifier (slow login), a subject with an adjectival complement (the app is useless), and a negated verb (the OTP never arrives → otp → not arrive; I can't renew my Emirates ID → emirates id → cannot renew). Subject-verb-object triples work as a simple relation extractor, and Matcher patterns catch recurring complaint phrases (can't login, error). I also count unigrams, bigrams and trigrams per sentiment class.

4.3 Text representations

I compare four representations with the same logistic-regression classifier: bag of words, TF-IDF with unigrams and bigrams, Word2Vec skip-gram [17] trained on my reviews (three settings of vector size and window, with the review vector as the mean of its word vectors), and pre-trained BGE-M3 sentence embeddings (1024 dimensions, multilingual). Besides accuracy and macro-F1 I report precision@5, the share of a review's five nearest neighbours that have the same label. For the cross-lingual test I ask five questions in English and in Arabic and measure how many of the top-10 results are the same.

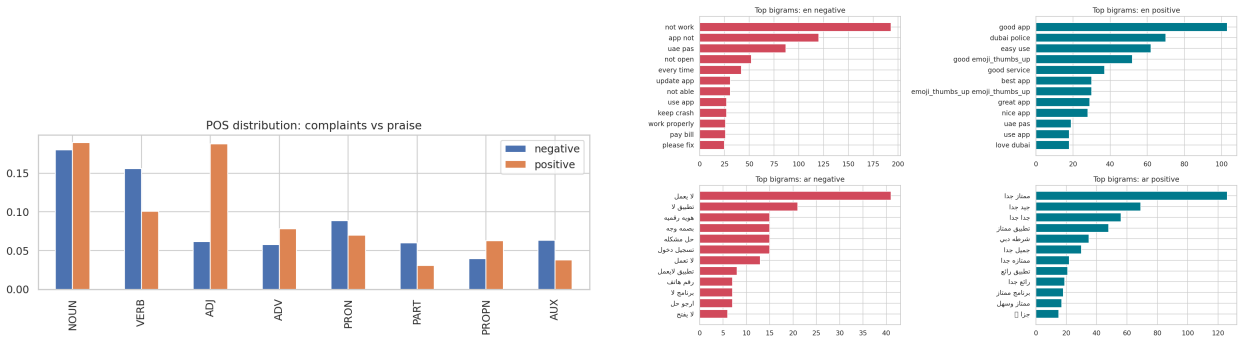


Figure 2: Left: POS distribution in complaints and praise. Right: most frequent bigrams per language and class.

4.4 Sentiment, fine-tuning and emotion

I evaluate five sentiment approaches on the same test set: VADER (lexicon, English only), TF-IDF with logistic regression trained on my data, a pair of pre-trained models chosen by language (Twitter-RoBERTa for English and CAMeLBERT-DA sentiment for Arabic), the multilingual XLM-T sentiment model, and XLM-R base fine-tuned with LoRA (rank 16 on the attention query and value projections plus the classification head, 0.42% of the parameters trainable, 3 epochs, AdamW, learning rate 2×10^{-4}, class-weighted cross-entropy and mixed precision, 1.1 minutes on a T4 GPU). For emotions I use Ekman's categories plus neutral [5]: a fine-tuned DistilRoBERTa [8] for English and zero-shot NLI with mDeBERTa [14] for Arabic, with the same labels. I then place each emotion on Russell's valence-arousal circumplex [19]. To decide which reviews need a reply first, I defined a simple urgency score:

$$U = 100\left(0.45\,p_{\text{neg}} + 0.35\,\min(1,\,p_{\text{anger}} + p_{\text{disgust}} + p_{\text{fear}}) + 0.20\,s\right) + 10\cdot\mathbb{1}[\text{critical topic}], \quad (1)$$

where $s = \min\big(1, \log(1 + \text{thumbs-up})/\log 51\big)$ measures how many other users agree, and the critical topics are money, fines, visas and account lock-outs (with English and Arabic keywords).

4.5 Named entities and relations

I compare spaCy NER (English) and CAMeLBERT-mix NER (Arabic) with an LLM that I prompt to return JSON with the app feature, an issue type, typed entities (including domain types such as GOV_SERVICE and ERROR_MESSAGE) and (subject, relation, object) triples. Reviews are sent in small batches to stay within the free-tier limits.

4.6 Issue discovery and release regressions

I cluster the complaints (1 or 2$\star$, or predicted negative) with k-means on the BGE-M3 vectors and choose k with the silhouette score. Each cluster is described by class-based TF-IDF keywords [6] and the eight reviews closest to its centre, and the LLM uses these to write an English and an Arabic name. For each app version with at least 20 reviews, a two-proportion z-test compares its complaint rate p_1 with the rest of the app's reviews p_2:

$$z = \frac{p_1 - p_2}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}}. \quad (2)$$

Because 65 versions are tested at the same time, about one in twenty would reach $p < 0.05$ just by chance. So I convert z to a one-sided p-value and apply the Benjamini-Hochberg procedure, which keeps the false discovery rate (FDR) at 5%: with the p-values sorted, $q_{(i)} = \min_{j \geq i} p_{(j)}\, m/j$. A version is flagged when $q < 0.05$ and $p_1 - p_2 \geq 0.10$, and the cluster whose share grew the most shows what broke.

4.7 Prompt engineering, RAG and faithfulness

I used free models through OpenRouter (dots-studio/dots-3-note-preview:free for the classification experiment). Three prompts classify the same balanced sample: zero-shot, a role with label definitions, and few-shot with bilingual examples. The Ask Citizen Voice assistant retrieves the 12 most relevant reviews with BGE-M3, answers only from them in the language of the question with citations (`[R3]`), and a second prompt acts as a judge that splits the answer into claims and checks each one against the cited reviews [22]. I compare this with the same question answered without retrieval. Replies to citizens are written in English and Arabic with clear constraints (no promises and no invented contact details).

5 Results

Table 3: Text representations (same logistic-regression classifier, test set).

Representation	Dim.	Accuracy	Macro-F1	Precision@5
BoW	1819	0.843	0.611	0.771
TF-IDF (1-2 grams)	3622	0.829	0.6	0.787
Word2Vec (trained here)	50	0.786	0.584	0.792
BGE-M3 (pre-trained)	1024	0.833	0.633	0.866

Table 4: Cross-lingual retrieval with parallel English and Arabic queries (mean over 5 pairs).

Method	Overlap@10	Arabic share for English query
BGE-M3	0.36	0.2
TF-IDF	0	0

Table 5: Sentiment analysis on the test set (star labels as ground truth).

Method	Accuracy	Macro-F1	Macro-F1 (EN)	Macro-F1 (AR)
VADER (lexicon)	–	–	0.532	–
TF-IDF + LR	0.829	0.6	0.623	0.558
RoBERTa / CAMeLBERT (pre-trained)	0.843	0.639	0.639	0.602
XLM-R (pre-trained)	0.857	0.643	0.64	0.632
XLM-R + LoRA (fine-tuned)	0.899	0.615	0.613	0.606

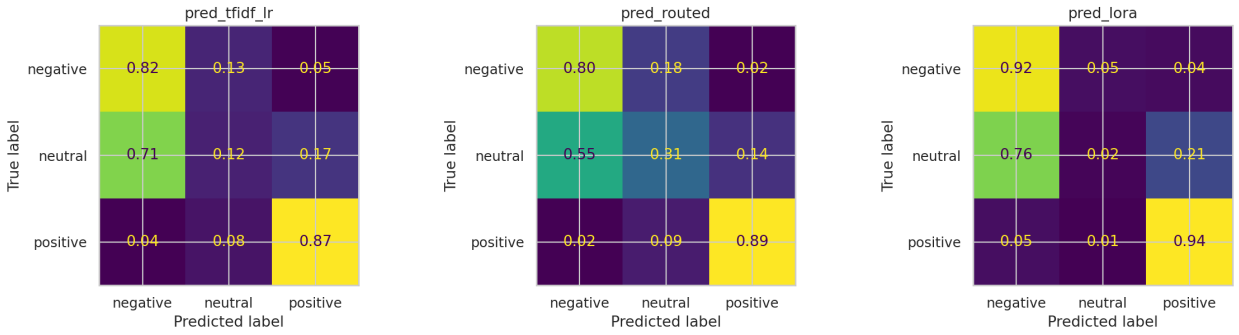


Figure 3: Normalised confusion matrices: TF-IDF with LR, routed pre-trained models, LoRA fine-tuned XLM-R.

Table 6: Prompt engineering on a balanced sample (same LLM, different prompts).

Strategy	Accuracy	Macro-F1
zero_shot	0.66	0.569
role_definitions	0.66	0.594
few_shot	0.7	0.655
fine-tuned XLM-R + LoRA (same reviews)	0.64	0.545

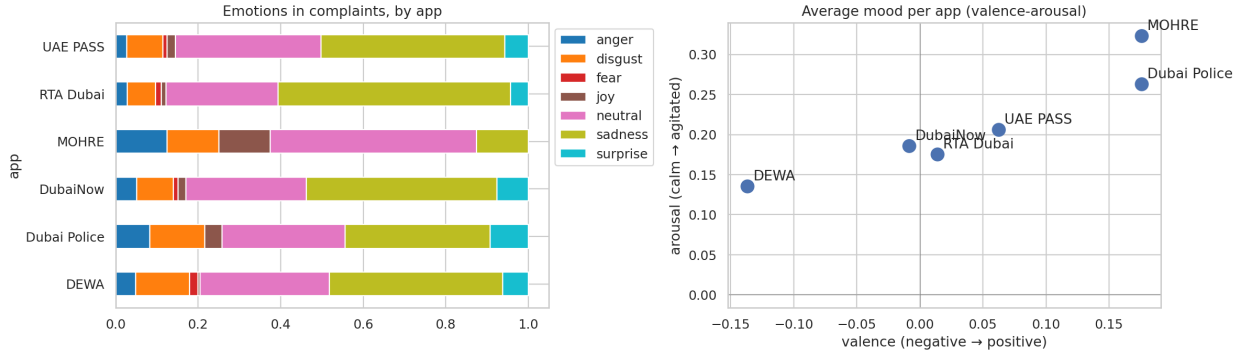


Figure 4: Emotions in complaints by app, and average mood on the valence-arousal plane.

Table 7: Largest automatically discovered issues (13 clusters in total).

Issue (LLM name)	Complaints	Avg. rating	Share Arabic
Parking payment issue	200	1.65	0.15
Poor customer service	184	1.20	0.16
Payment failure	172	1.45	0.12
Login problem	161	1.36	0.43
App launch failure	160	1.41	0.32
Update malfunction	144	1.53	0.14
App crash on update	127	1.54	0.07
App not working	124	1.46	0.44

Table 8: Flagged release regressions after the Benjamini-Hochberg correction (10 of 65 versions).

App	Version	Reviews	Complaint rate	Baseline	z	q
DEWA	10.3.6	20	0.95	0.511	3.87	0.001
Dubai Police	7.0.5	25	0.2	0.069	2.51	0.039
RTA Dubai	6.2.7	33	0.515	0.287	2.82	0.022
RTA Dubai	6.2.9	42	0.5	0.286	2.98	0.019
RTA Dubai	6.3.6	40	0.475	0.287	2.55	0.039
RTA Dubai	6.4.2	43	0.628	0.28	4.89	0
RTA Dubai	6.4.3	24	0.542	0.289	2.68	0.03
RTA Dubai	6.4.6	33	0.515	0.287	2.82	0.022
RTA Dubai	6.4.8	27	0.778	0.281	5.58	0
UAE PASS	6.9.1	142	0.352	0.227	3.20	0.011

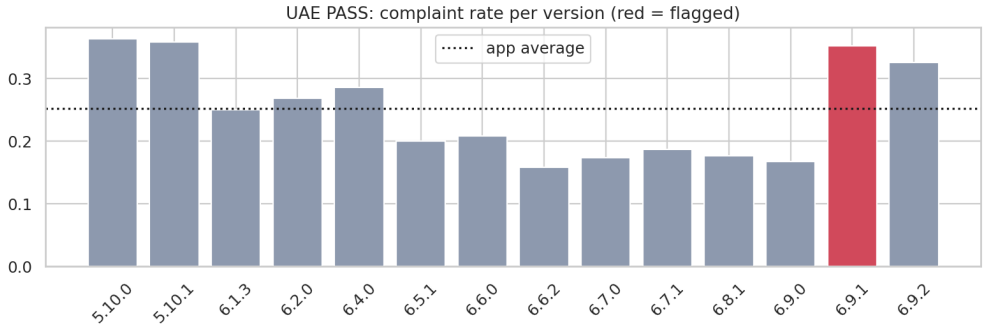


Figure 5: Complaint rate per version for the most reviewed app (red: flagged).

Table 9: RAG assistant: faithfulness of the answers (share of supported claims) with and without retrieval.

Question	With RAG	Without retrieval	Citations
Why are people unhappy with UAE PASS login?	–	0.57	12
What payment problems do users report?	1.00	0.57	12
(Arabic)	–	–	9

6 The Citizen Voice Application

All of this is available to an app team in a Streamlit application that I run from Colab with a public link. It has seven tabs: Overview (key numbers, sentiment by app, weekly complaint trend and an LLM executive briefing), Issues (the bilingual clusters, aspect and opinion pairs and frequent bigrams), Releases (the regression detector per app and version), Search (cross-lingual semantic search with BGE-M3), Ask Citizen Voice (the RAG assistant with citations and the faithfulness check), Triage and Reply (reviews ranked by urgency with a bilingual reply draft) and Analyze a review, which shows every step for a new review: cleaning, tokens, stems, lemmas, POS tags, the dependency tree, entities, sentiment, emotion and urgency. A sidebar button collects new reviews and re-runs the pipeline, so only the new reviews are analysed. The pipeline also exports a one-page HTML dashboard of the results

(Figure 6) that can be opened in any browser.

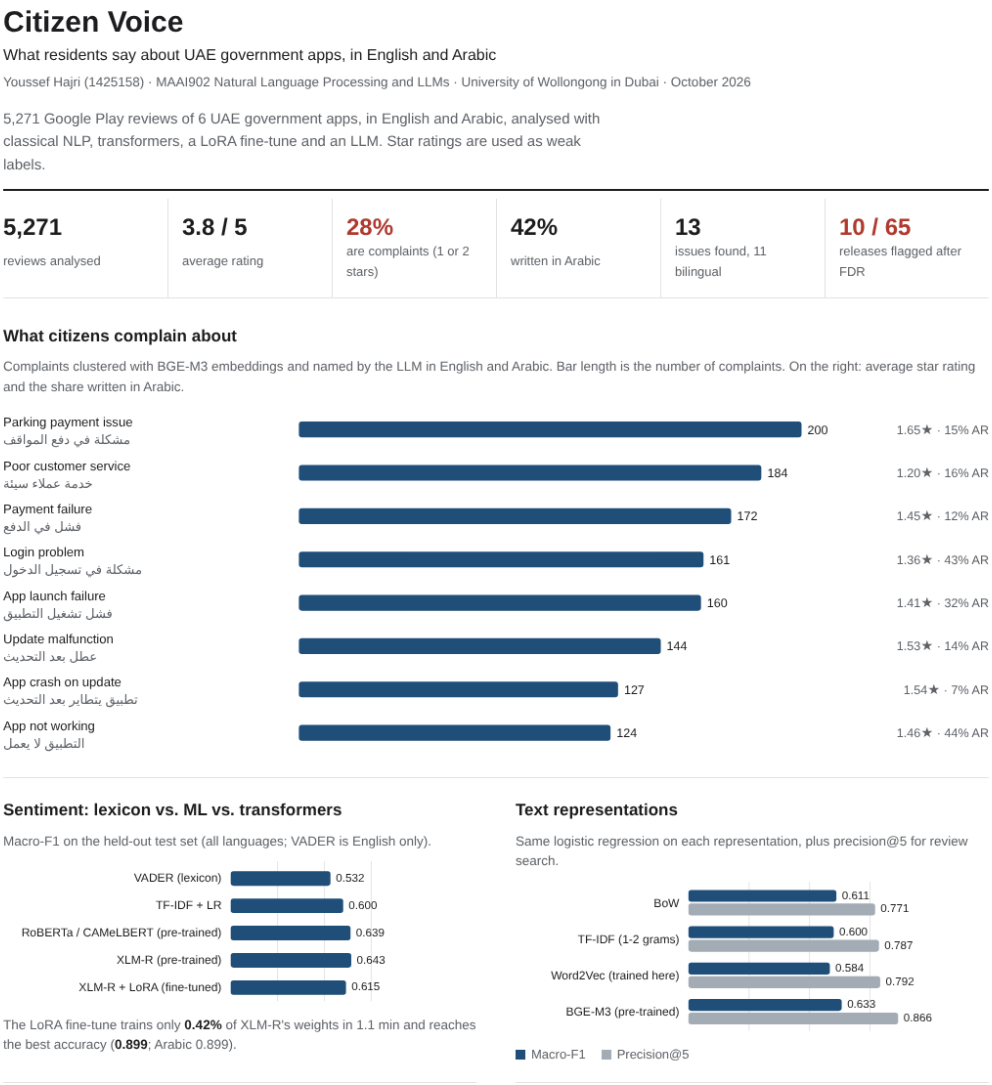


Figure 6: Top of the results dashboard exported by the pipeline.

6.1 The application in use

Figures 7 to 12 show the running application. I took these screenshots after the presentation, from a new run of the collector that had grown the data to 6,428 reviews. The numbers on the screens are therefore a little different from the tables in Section 5, which come from the evaluation run, but the application and the methods are the same. The sidebar on every page lets the user filter by app, language and date, shows that the LLM is connected, and has the button that collects new reviews.



Figure 7: Overview tab: key numbers, predicted sentiment per app and the emotions in complaints.

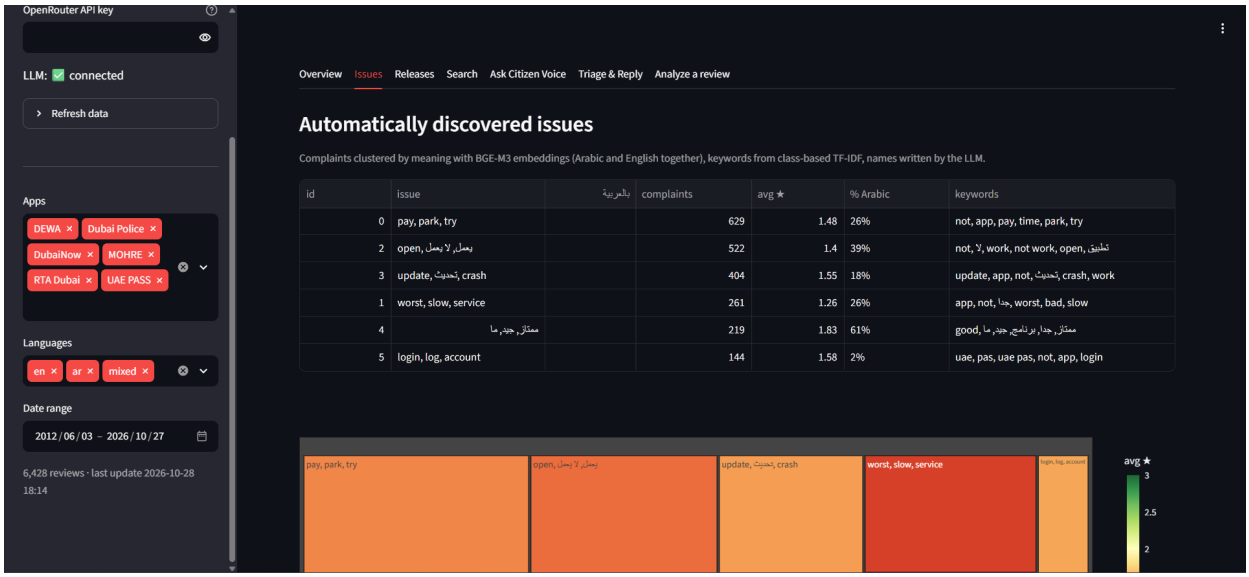


Figure 8: Issues tab: complaints clustered with BGE-M3, with the share of Arabic reviews and the class-based TF-IDF keywords of each issue.

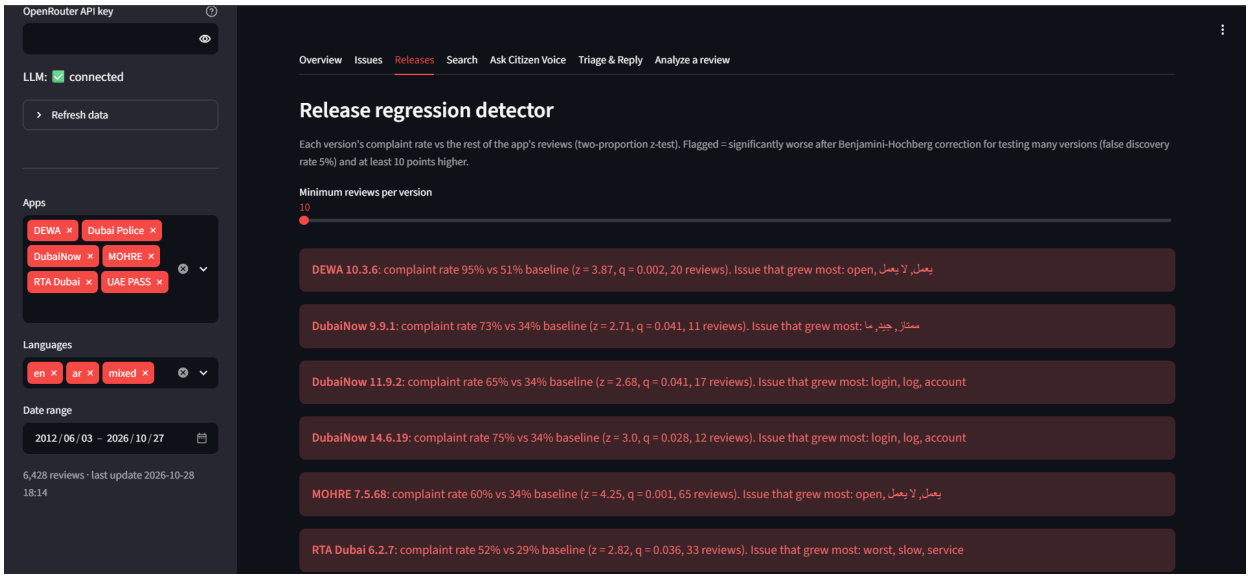


Figure 9: Releases tab: versions flagged by the two-proportion z-test after the Benjamini-Hochberg correction, with the issue that grew the most.

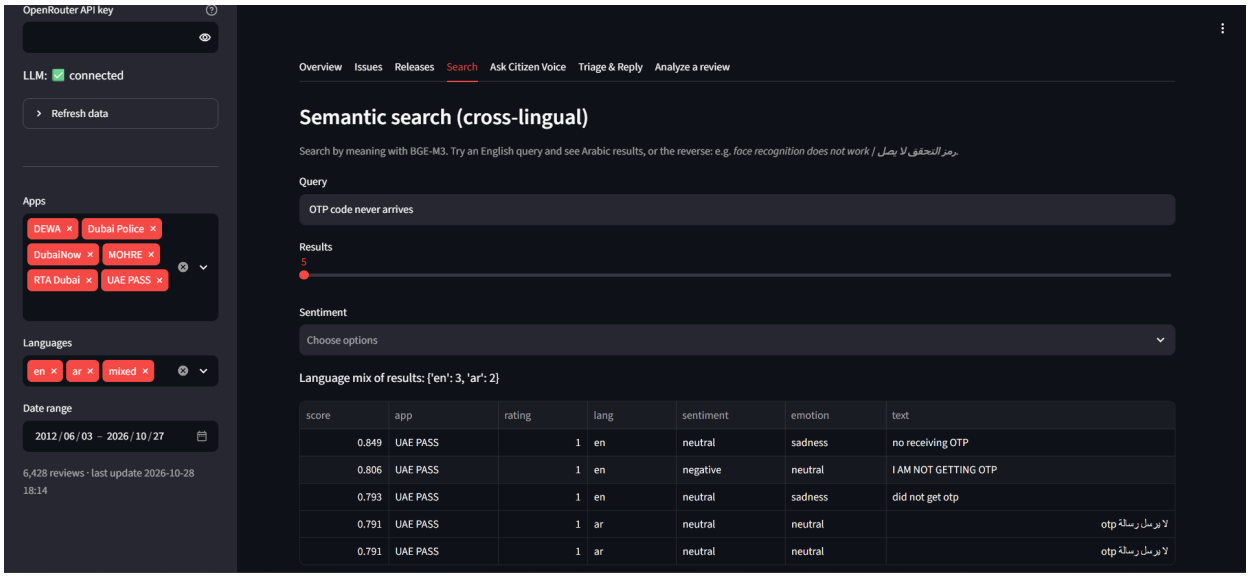


Figure 10: Search tab: the English query "OTP code never arrives" returns English and Arabic reviews about the same problem.

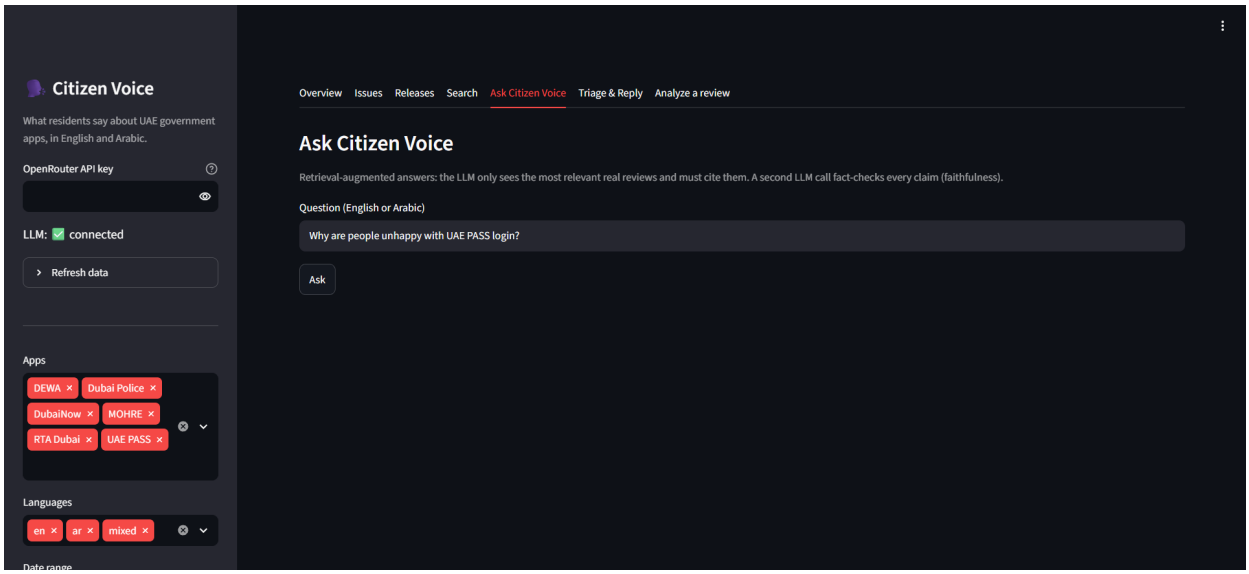


Figure 11: Ask Citizen Voice tab: a question in English or Arabic is answered by the RAG assistant from the retrieved reviews, with citations and a faithfulness check by a second LLM call.

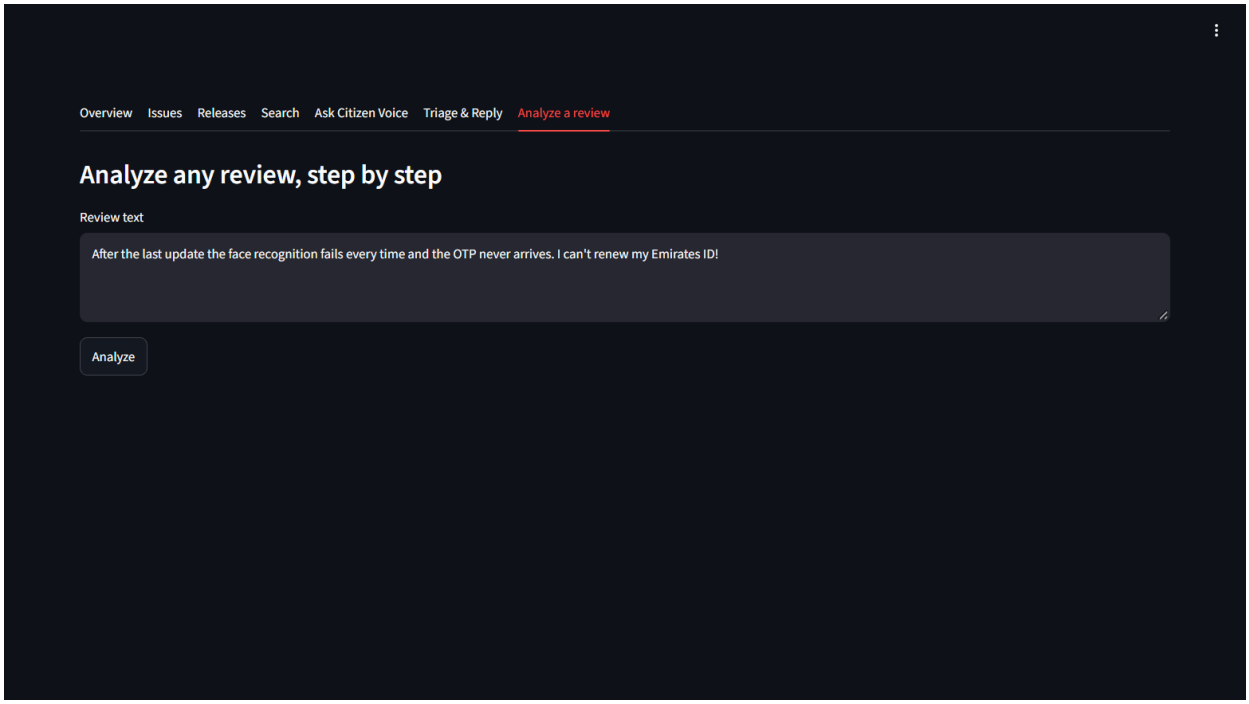


Figure 12: Analyze a review tab: any new review can be analysed step by step, from cleaning and tokens to entities, sentiment, emotion and urgency.

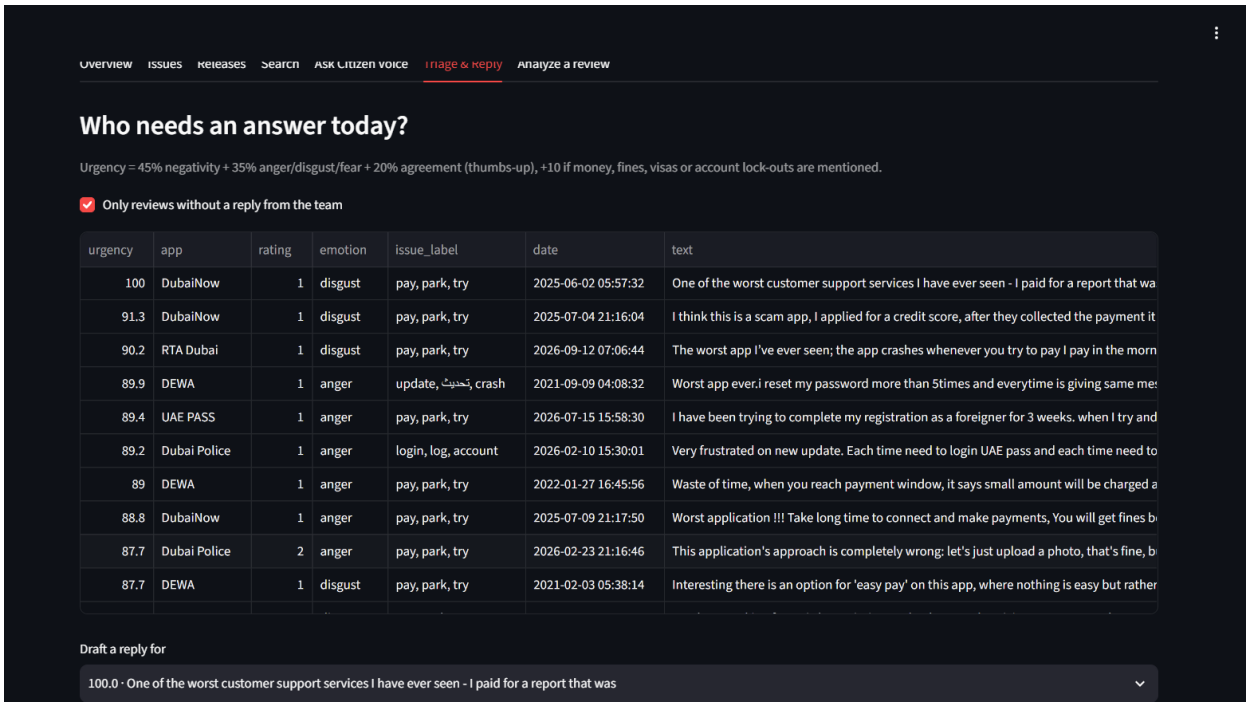


Figure 13: Triage and Reply tab: unanswered reviews ranked by the urgency score, ready for a bilingual reply draft.

7 Discussion

Representations. With the same classifier, TF-IDF gets the highest accuracy (0.829), because a few keywords (useless, worst, mumtāz) already decide most polar reviews. BGE-M3 gets the best macro-F1 (0.633 against 0.6 for TF-IDF) and the best neighbourhood quality (precision@5 0.866), so it deals better with the small neutral class and with paraphrases. Word2Vec trained on my small corpus of very short reviews is the weakest (macro-F1 0.584), since averaging a few word vectors loses word order and negation. The biggest difference shows up across languages. TF-IDF has no shared vocabulary between the two scripts, so an English query never finds an Arabic review (overlap 0). BGE-M3 returns on average 0.36 of the same top-10 reviews for the English and Arabic version of a question, and 20% of the results for an English query are Arabic reviews. This is the reason I built search, issue discovery and RAG on BGE-M3.

Sentiment and fine-tuning. VADER is clearly the weakest (English macro-F1 0.532). It misses domain phrases such as the OTP never arrives, which contain no negative word, and it cannot read Arabic. The pre-trained transformers beat the TF-IDF classifier on macro-F1 (0.643 for XLM-T and 0.639 for the RoBERTa and CAMeLBERT pair, against 0.6) without seeing any of my reviews. Fine-tuning XLM-R with LoRA, with only 0.42% of its parameters trained for 1.1 minutes, gives the best overall accuracy (0.899) and the best Arabic accuracy (0.899, Arabic macro-F1 0.606), but not the best overall macro-F1 (0.615 against 0.643). I think the reason is the neutral class. The fine-tuned model learns the large polar classes very well, but three epochs on a few hundred 3-star reviews, with a classification head that starts from zero, are not enough to learn "neutral", while XLM-T was already trained on a three-class sentiment task. For me this was the clearest example of pre-training against fine-tuning in the project: fine-tuning wins where there is enough in-domain data (polar and Arabic reviews), and the knowledge from pre-training wins where data is scarce. Training for more epochs, or starting LoRA from the XLM-T checkpoint instead of plain XLM-R, would be my next steps.

Emotion. Most complaints are sad (46% of 1 and 2-star reviews) and very few are angry (4%). People sound disappointed ("not working", "can't log in") more than furious, and many very short reviews are labelled neutral. On the valence-arousal plane the complaints sit at negative valence but low arousal. For triage this suggests giving sadness more weight in the urgency score, because a calm complaint can still describe a blocking problem.

Issues and releases. Clustering the complaints gives 13 issues, and 11 of them contain a real share of both Arabic and English reviews, so the same problem is tracked whatever language people write in. The LLM turns each cluster's keywords and examples into a short bilingual name (Table 7), and the issues are concrete: parking payments, failed card payments, login problems, the app not opening, and crashes after updates. The "Login problem" cluster is 43% Arabic, which shows that registration and identity complaints in both languages end up in the same issue. The aspect rules point the same way: the most frequent English pairs are app $\rightarrow$ not working, login $\rightarrow$ not working and otp $\rightarrow$ cannot receive. For releases, I tested 65 app versions and 15 of them have $p < 0.05$ on their own. After the Benjamini-Hochberg correction 10 remain (Table 8), and most of them are consecutive RTA Dubai releases, which looks like a lasting problem and not one bad update. In the strongest cases the complaint rate jumps from under 30% to over 60%.

Prompting and grounding. On the same balanced sample, changing only the prompt moves the LLM from macro-F1 0.569 (zero-shot) to 0.594 with a role and label definitions, and to 0.655 with five bilingual few-shot examples, without any training. So the prompt really is part of the model. Few-shot prompting even beats the fine-tuned XLM-R on these

50 reviews (0.545), mostly because the examples show clearly what "neutral" means. The sample is small, so I see this as an indication and not a final ranking. For classifying thousands of reviews the fine-tuned model is still the practical choice, since it is free, fast and always gives the same answer. The LLM is more useful for generation: naming clusters, extracting relations, drafting replies and answering questions. For NER, the LLM found fewer entities per review than spaCy and CAMeLBERT together (0.27 against 0.64), but more useful ones for this domain, for example Salik as an organisation or account is locked as an error message, while the classical models mostly find cities, dates and numbers. Grounding made a clear difference for the answers. Without retrieval only 57% of the LLM's claims were supported by the reviews, while the RAG answer with citations was fully supported where the judge gave a verdict (Table 9), and every citation pointed to a real retrieved review. The free models did not always return valid JSON for the judge, which is why some cells are empty.

8 Limitations and Ethics

Star ratings are only a weak label, and a small set annotated by hand would give a cleaner evaluation. The very short reviews limit every method. Some of the apps I targeted could not be matched reliably on the store (TAMM was not found, and the ICP and MOHRE searches returned other apps with almost no reviews), so the data mostly comes from five large Dubai and federal apps. CAMeLBERT-NER tags religious expressions such as al-ḥamdu lillāh ("thank God") as entities (Allah, MISC), which shows that general NER models need filtering for a domain like this. My dependency rules and spaCy NER only work for English, and Arabic dialect spelling and Arabizi are still hard. Arabic emotion relies on zero-shot NLI, which is less accurate than a dedicated model. Free LLM endpoints change over time, so the LLM results can vary between runs (I cache the answers so they can be reproduced). Reviews can contain personal information, so I store no names, show only aggregates, and the generated replies have to be checked by a person before they are posted.

9 Conclusion

With Citizen Voice I tried to show how the whole NLP toolbox from the course, from tokenisation and stemming to fine-tuned transformers and grounded LLMs, can be combined into one bilingual system that is useful for smart government in the UAE. If I continue the project, I would add an Arabic dependency parser for Arabic aspects, a test set labelled by hand, an Arabic LLM such as Falcon-Arabic for the replies, and alerts that notify the app team automatically when a regression is detected.

References

[1] Francesco Barbieri, Luis Espinosa Anke, and Jose Camacho-Collados. XLM-T: Multilingual language models in Twitter for sentiment analysis and beyond. In *Proceedings of LREC*, 2022.

[2] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024.

[3] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, et al. Unsupervised cross-lingual representation learning at scale. In *Proceedings of ACL*, 2020.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.

[5] Paul Ekman. An argument for basic emotions. *Cognition and Emotion*, 6(3-4):169 to 200, 1992.

[6] Maarten Grootendorst. BERTopic: Neural topic modeling with a class-based TF-IDF procedure. *arXiv preprint arXiv:2203.05794*, 2022.

[7] Emitza Guzman and Walid Maalej. How do users like this feature? a fine grained sentiment analysis of app reviews. In *IEEE International Requirements Engineering Conference (RE)*, 2014.

[8] Jochen Hartmann. Emotion english distilroberta-base. `https://huggingface.co/j-hartmann/emotion-english-distilroberta-base`, 2022.

[9] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength natural language processing in Python, 2020.

[10] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[11] C. J. Hutto and Eric Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, 2014.

[12] Go Inoue, Bashar Alhafni, Nurpeiis Baimukan, Houda Bouamor, and Nizar Habash. The interplay of variant, size, and task type in Arabic pre-trained language models. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*, 2021.

[13] Leah S. Larkey, Lisa Ballesteros, and Margaret E. Connell. Light stemming for Arabic information retrieval. In *Arabic Computational Morphology*, page 221 to 243. Springer, 2007.

[14] Moritz Laurer, Wouter van Atteveldt, Andreu Casas, and Kasper Welbers. Less annotating, more classifying: Addressing the data scarcity issue of supervised machine learning with deep transfer learning and BERT-NLI. *Political Analysis*, 32(1):84 to 100, 2024.

[15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[16] Daniel Loureiro, Francesco Barbieri, Leonardo Neves, Luis Espinosa Anke, and Jose Camacho-Collados. TimeLMs: Diachronic language models from Twitter. In *Proceedings of ACL: System Demonstrations*, 2022.

[17] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[18] Ossama Obeid, Nasser Zalmout, Salam Khalifa, Fadhl Eryani, Alexander Erdmann, Kemal Oflazer, and Nizar Habash. CAMeL tools: An open source python toolkit for Arabic natural language processing. In *Proceedings of LREC*, 2020.

[19] James A. Russell. A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6):1161 to 1178, 1980.

[20] Kazem Taghva, Rania Elkhoury, and Jeffrey Coombs. Arabic stemming without a root dictionary. In *International Conference on Information Technology: Coding and Computing (ITCC)*, 2005.

[21] UAE Cabinet. UAE national strategy for artificial intelligence 2031. `https://u.ae/en/about-the-uae/strategies-initiatives-and-awards/strategies-plans-and-visions/government-services-and-digital-transformation/uae-strategy-for-artificial-intelligence`, 2017.

[22] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks*, 2023.